OpenAgenet / OAN White Paper: Open Infrastructure for Trusted Agent Interconnection

Jinliang Xu
China Academy of Information and Communications Technology, Beijing, China
xujinliang@caict.ac.cn; jlxufly@gmail.com

***Abstract*—OpenAgenet, abbreviated as OAN, is an open infrastructure project for trusted Agent interconnection. It addresses a problem that becomes visible when Agents move from isolated applications into open, multi-operator networks: before an Agent can safely discover, select, and invoke another Agent, it needs a way to verify identity provenance, governance state, discovery authorization, freshness, and pre-connection trust evidence. OAN is designed as a protocol-neutral trust layer. It does not replace Agent interaction protocols, tool protocols, model orchestration frameworks, or application-level workflows. Instead, it provides routing-aware `did:oan`-based resource identity, local OAN Identity material for signing and authentication, governance-backed admission, Registrar-assisted onboarding, Root-verified package publication, authorization-aware Discovery, Root-issued infrastructure authorization VCs, controller-aware authorization, and signed trusted invocation. The architectural center of OAN is the combination of federated governance, resource identity, trusted Discovery, and the Root Node's three hub responsibilities. Root acts as a trust authorization hub for accepted identity state and evidence, a data-distribution hub for Root-verified packages, and a semantic-governance hub for the tag tree shared by registration and Discovery. This white paper explains the motivation, architecture, roles, governance model, relationship with MCP, A2A, and ANP, deployment patterns, cooperation model, on-chain governance layer, prototype status, public pilot network, standardization surfaces, performance profile, and roadmap of OAN.**

Public portal: https://www.openagenet.xyz/
Open-source project: https://github.com/OpenAgenet

***Index Terms*—OpenAgenet, OAN, Agent Internet, did:oan, discoverable resource, trusted discovery, governance, MCP, A2A, ANP**

1. Executive Summary

Large language model based Agents are changing from local assistants into network participants. They call tools, consume data, expose capabilities, delegate work, negotiate task context, and cooperate with other Agents. As this shift continues, the central problem is no longer only how one Agent sends a message to another Agent. A prior infrastructure question must be answered: when an Agent is discovered in an open environment, who says that this Agent is the one it claims to be, who accepted its identity, whether the accepted identity is still current, whether the Discovery service is authorized to expose it, and how a relying Agent can validate these facts before interaction.

OAN addresses this pre-connection trust problem. It defines an open infrastructure architecture for routing-aware `did:oan`-based resource identity registration, governance-backed Root acceptance, trusted package distribution, authorization-aware Discovery, controller-aware authorization, and signed invocation. The design is intentionally layered. OAN focuses on identity, lifecycle governance, discoverability, and trust evidence. It leaves task semantics, tool protocols, model orchestration, dialogue strategy, and application workflow to protocols and frameworks that operate above it.

OAN separates three identity artifacts that are often conflated. The DID Document is the public identity document. OAN Identity is local private identity material that binds a DID, a DID Document, public key material, and private key material for signing. A VC is a signed issuer statement about an authorization or registration fact. This separation allows a resource to be self-controlled, controlled by a dedicated controller, or controlled by an external controller with explicit authorization proof, without turning the controller string itself into a trust shortcut.

The same separation also shapes application authentication. A user or resource can import an OAN Identity into an application and sign a fresh challenge, but the application should verify the signature against Root-accepted public-key evidence rather than against an arbitrary user-supplied DID Document. In this model, OAN Identity is the local signing material, the DID Document is the public key and service declaration, and Root-accepted identity projections help applications discover which public key should be trusted for a login decision.

This distinction matters for the trust model. Root is not intended to be a silent single point that alone decides infrastructure legitimacy. In the upgraded architecture, infrastructure lifecycle decisions for Registrars, Discovery nodes, and future VC issuers are published by an on-chain governance layer that can be operated under committee and threshold rules. Root consumes that governance state, enforces it in service admission, and converts approved state into protocol-level authorization VCs. Root is therefore closer to a policy enforcement and credential issuance point than to the ultimate social governance authority.

This positioning can be summarized as **trusted resource registry and discovery infrastructure**. Many adjacent systems begin from a narrower primary object: an MCP registry centers on tool or server endpoints, a Skill marketplace centers on Skill artifacts, an Agent directory centers on Agent metadata, and an Agent naming system centers on names. OAN instead treats Agent Services, Skills, MCP Servers, and Tool/API endpoints as first-class discoverable resources whose identities, lifecycle, packages, and discovery responses must

be verifiable. This "Resource First" stance is the reason OAN can support Agent-centric ecosystems today while still leaving room for future resource forms such as workflows, prompts, datasets, or model endpoints.

The standardization implication is important. If the ecosystem standardizes only Agent records, it may later need separate registries for Skills, tools, model endpoints, and workflow assets. If it standardizes only tool registries, it cannot express lifecycle governance for non-tool resources. OAN therefore places the ResourcePackage at the center: a resource can keep its native artifact or protocol, while OAN standardizes the identity, provenance, registration evidence, lifecycle status, package proof, and discovery evidence around it. This is similar in spirit to artifact distribution systems such as OCI, where different artifacts can be distributed through a common manifest and registry discipline [1], [2]; OAN applies a trust-governed version of that idea to Agent Internet resources.

The project is motivated by a practical observation. A future Agent ecosystem will not be operated by one platform. Agents will be developed by enterprises, research institutes, individual developers, public-service systems, industrial platforms, and vertical-domain operators. They will run behind different gateways, use different runtimes, and support different interaction protocols. If every operator defines its own identity admission, directory, credential, and trust-verification mechanism, the ecosystem fragments quickly. OAN provides a common trust substrate that allows heterogeneous Agent Services, Skills, MCP Servers, and Tool/API resources to become discoverable and verifiable under shared governance rules.

OAN can be summarized through five commitments:

- **Identity before interaction**: a discoverable resource should have a governed digital identity before it becomes visible to other parties.
- **Lifecycle before directory**: Discovery should expose only Root-accepted, current, and policy-admissible identity versions.
- **Authorization before exposure**: a Discovery service should only expose resource identities within its authorized domains.
- **Verification before invocation**: a relying party should verify discovery provenance and request signatures before business interaction.
- **Compatibility before replacement**: OAN should complement MCP, A2A, ANP, DID/VC methods, and domain protocols instead of replacing them.

The result is a governance-oriented Agent interconnection layer. It gives different organizations a way to cooperate without forcing them into one Agent runtime, one model vendor, one application framework, or one task protocol. The strongest contribution is not any isolated object such as an identifier, directory, or package format. It is the coupling of federated infrastructure governance, resource identity, verifiable distribution, authorization-aware Discovery, and pre-connection verification. This structure is reflected in the public-facing adoption model: OAN supports four resource forms, three participant groups, DID-based semantic discovery, blockchain-backed federated governance, and distinct interaction surfaces for publication, Discovery, network transparency, documentation, SDK integration, and DID-method review.

The corresponding architectural emphasis is **Trust First**. OAN is not only a search layer. It combines the functions that, in the traditional Internet, would be spread across naming, certificate issuance, artifact distribution, and directory lookup. Discovery helps a relying party find candidates, Root and VC evidence help it decide whether those candidates are admissible, and the ResourcePackage gives it a verifiable distribution object. Within that emphasis, Root plays three distinct hub roles: it anchors trust evidence, distributes accepted resource state as verifiable packages, and governs the semantic tag tree that helps Registrars guide metadata and helps Discovery nodes interpret resource intent.

This also explains why OAN should integrate with, rather than replace, specialized communities. Schema communities can improve how resources describe themselves. Directory communities can improve routing, indexing, and scale. Naming communities can improve human-friendly aliases. Runtime communities can improve execution. OAN's contribution is the trust envelope that lets those outputs become safe to register, distribute, discover, and approach.

A. Contributions and Reading Guide

The white paper makes six contributions at the infrastructure level. First, it defines OAN as a trust layer for open Agent interconnection rather than as another Agent runtime or application protocol. Second, it generalizes the identity object from an Agent record into a resource-oriented `did:oan` model that can cover Agent Services, Skills, MCP Servers, and Tool/API endpoints. Third, it clarifies the separation among public DID Documents, local OAN Identity material, VCs, and controller authorization proofs. Fourth, it separates social governance, Root enforcement, Registrar onboarding, package distribution, Discovery exposure, and Agent invocation into distinct roles. Fifth, it explains how authorized-domain governance and Root-issued evidence constrain Discovery before invocation. Sixth, it connects the architecture to public interaction and implementation surfaces so that the proposal can be inspected, implemented, and discussed beyond a conceptual diagram.

Readers interested in motivation can focus on Sections 2–4. Readers interested in the trust path can focus on Sections 5–8. Readers interested in deployment, governance, and security can focus on Sections 9–12. Readers interested in public status, standardization, cooperation, and adoption can focus on the later sections. The technical yellow paper, protocol specifications, and conformance artifacts should be used for wire-level formats, state machines, validation algorithms, and implementation details.

2. Background: From Local Agents to Open Agent Networks

Early Agent systems were often embedded inside one application. The application owner controlled the model, prompt,

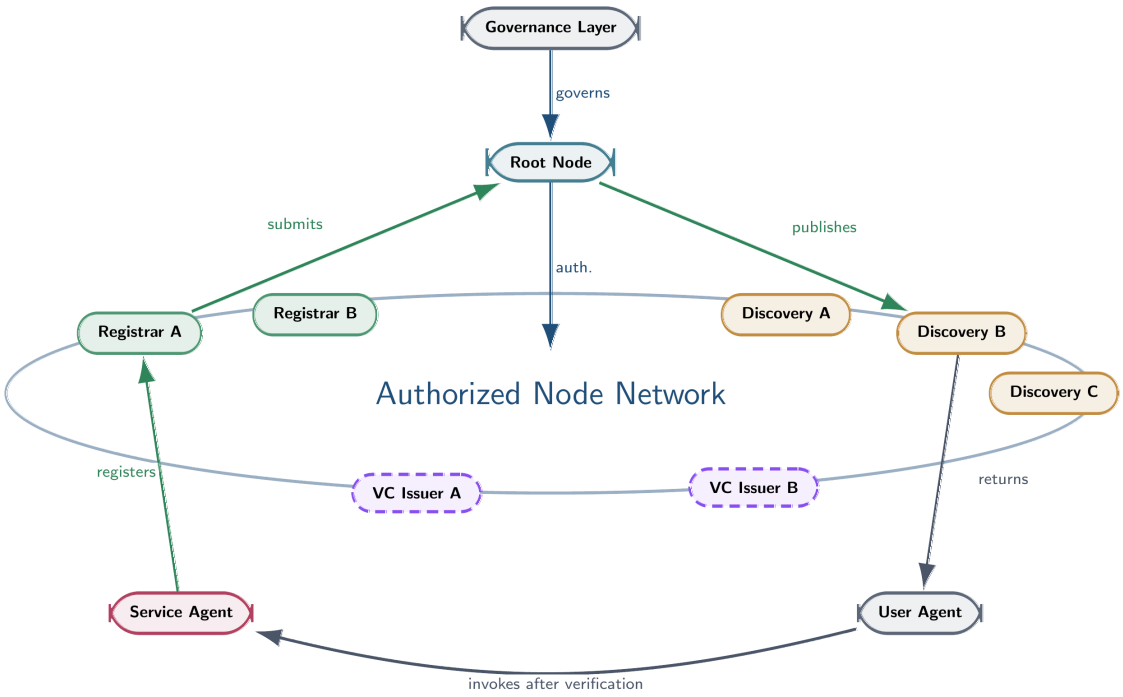


Fig. 1. OpenAgenet / OAN framework overview. OAN separates governance, Root acceptance, Registrar onboarding, Discovery exposure, VC issuance, and verified invocation into distinct roles within an authorized node network.

tool list, user interface, permissions, and execution environment. In that setting, identity and discovery were simple because the platform boundary was clear. The application already knew which components it trusted.

Open Agent interconnection changes this assumption. A User Agent may need to find a professional Service Agent operated by another organization. A Service Agent may rely on tools exposed through MCP servers. A workflow platform may hand a task to an A2A-compatible Agent. A public-domain directory may publish Agents that serve public services, industry data, or enterprise operations. Recent multi-agent systems and surveys show the same shift from isolated Agent prototypes toward networked, tool-using, and collaborative Agent workflows [3], [4], [5], [6]. This open setting creates four infrastructure gaps.

A. Identity Gap

An Agent needs an identity that is more than a display name, endpoint URL, or marketing description. It needs verifiable subject information, verification methods, service endpoints, capability metadata, and lifecycle state. The resource identity should be signed, versioned, and traceable to a governance decision. Without such a representation, relying parties cannot distinguish an accepted Agent from an imitation, stale copy, or unreviewed endpoint.

B. Governance Gap

Open networks need a way to decide which infrastructure participants can register Agents, which Discovery services can expose which domains, and which resource identity versions are accepted. Governance is not only a social process. It must be reflected in machine-verifiable records. Otherwise, operators must manually trust directory contents and cannot automate validation.

C. Discovery Gap

Discovery is not merely search. In an open Agent network, Discovery is a controlled exposure function. It should return candidates that match the query and also satisfy identity provenance, freshness, and authorization constraints. A Discovery node that is allowed to expose manufacturing Agents should not automatically expose medical, financial, or government-service Agents simply because it can fetch their metadata.

D. Invocation Gap

Even after discovery, a Service Agent should not accept every incoming request that names a known Agent. The request should be bound to caller identity, target identity, body hash, timestamp, nonce, and proof. This guard is especially important when Agents trigger actions, retrieve data, or initiate downstream tool calls.

OAN is designed to close these gaps while remaining small enough to be implemented by independent operators.

3. Why OAN, Why Now

The Agent ecosystem is entering a phase where interaction protocols are moving faster than trust infrastructure. MCP makes tools and data sources easier for AI applications to

access [7]. A2A-style protocols make Agent-to-Agent task exchange more explicit [8]. Agent DID methods and ANP-like efforts explore identity and networking conventions [9]. These developments are necessary, but they leave a common precondition unresolved: before an Agent enters a tool session, task session, or domain workflow, the relying party must know whether the Agent identity is governed, current, discoverable under policy, and verifiable.

Traditional approaches are not sufficient by themselves. API keys identify access relationships but do not provide open discovery or lifecycle provenance. Private directories work inside one platform but do not scale across organizations. Ordinary service registries expose endpoints but usually do not encode Agent subject control, governance acceptance, or Discovery authorization. DID documents and Verifiable Credentials provide important identifier and credential substrates [10], [11], but they do not by themselves define who may register an Agent into an ecosystem, which Discovery node may expose it, or how a relying Agent should validate pre-connection evidence.

OAN is timely because open Agent cooperation needs a layer that is neither a closed platform account system nor another task protocol. It is a trust infrastructure layer that can sit beside existing protocols and make them safer to use across organizational boundaries.

The problem is no longer limited to Agent service endpoints. Agent ecosystems are also beginning to publish Skills, MCP Servers, Tool/API endpoints, workflow components, prompts, datasets, and model-facing services. These resources are usually scattered across code repositories, portals, marketplaces, enterprise systems, and protocol-specific directories. A normal search engine or endpoint list can help a user find where something is, but it cannot by itself answer whether the resource is current, whether the publisher controls the claimed identity, whether a governance process accepted it, or whether a Discovery node is authorized to expose it. OAN turns this repeated application-level problem into shared trust infrastructure.

This shift also changes the unit of trust. In many early Agent systems, the main question was whether one Agent endpoint could be called. In open ecosystems, the harder question is whether many heterogeneous resources, operated by different organizations and discovered through different nodes, can share a common pre-connection trust path. OAN therefore treats trusted interconnection as a resource-network problem: resources need identities, governance evidence, lifecycle state, semantic visibility, and verification material before task-level protocols begin.

4. Vision and Scope

The long-term vision of OAN is an open Agent Internet in which Agents and other AI resource products can be registered, discovered, verified, and invoked across organizational boundaries under common trust rules. The project does not assume that the entire world will share one governance root. It assumes that each trust domain can combine a governance layer, a Root service, and authorized infrastructure nodes. Federation can happen inside a trust domain through committee-governed infrastructure lifecycle decisions, and future work can define cross-domain federation among multiple trust domains. The current scope is therefore a practical first step: build a clear governed trust-domain model, prove the identity lifecycle, support multiple Registrar and Discovery nodes, and expose enough interfaces for partners to integrate existing Agents.

A. What OAN Is

OAN is:

- a lifecycle model for governed AI resource identity;
- a resource-oriented DID method for Agent Services, Skills, MCP Servers, and Tool/API endpoints;
- a federated infrastructure governance model for Root, Registrar, Discovery, and future VC issuer roles;
- a package-distribution model for Root-verified identity artifacts;
- an authorization-aware Discovery model based on authorized domains;
- a pre-connection verification model for Agent-to-Agent invocation;
- a public pilot surface for observing, registering, discovering, and evaluating trusted resources;
- a reference implementation profile and developer-facing integration surfaces for ecosystem participation.

B. What OAN Is Not

OAN is not:

- a replacement for MCP, A2A, ANP, or other interaction protocols;
- a model runtime, LLM inference framework, or Agent-planning engine;
- a ranking algorithm or marketplace business model;
- a universal semantic ontology for all Agent capabilities;
- a centralized platform that must observe all Agent business traffic.

This boundary is important. OAN is deliberately positioned below application protocols and above raw cryptographic primitives. It gives other protocols a shared trust context, but it does not prescribe their internal task semantics. This white paper therefore does not freeze wire formats, API schemas, canonical serialization, or SDK function signatures. Those implementation details belong in the technical yellow paper, protocol specifications, and conformance artifacts. The purpose here is to define the value, roles, trust path, governance vocabulary, and adoption model that make those lower-level specifications meaningful.

5. Design Principles

A. Governance and Protocol Should Meet at Verifiable Facts

Real ecosystems require governance. Operators must authorize infrastructure nodes, review participants, define authorized domains, handle revocation, and audit misbehavior. At the same time, runtime systems require machine-verifiable facts.

OAN connects these two sides through signed records, Root-verified packages, bulletin facts, credentials, document hashes, and signed responses.

B. Discovery Should Be Policy-Aware

Traditional service discovery often focuses on name, address, or metadata matching [12], [13]. OAN treats discovery as an authorization-sensitive function. A candidate is not admissible merely because it exists and matches a keyword. It must be accepted by Root, current, valid under authorized-domain policy, and returned by an authorized Discovery node.

C. Compatibility Should Be Designed In

The Agent ecosystem already contains tool protocols, task protocols, DID methods, API gateways, service meshes, and enterprise identity systems. OAN should not require these systems to be discarded. It should provide adapters, metadata mapping, and pre-connection checks that allow existing protocols to benefit from governed identity and discovery.

D. Trust Should Be Distributed by Role

No single operational component should silently inherit all trust. The governance layer records infrastructure lifecycle decisions; Root enforces those decisions and issues protocol credentials; Registrar helps with onboarding but does not decide final acceptance; CDN distributes packages but does not become a trust authority; Discovery queries and indexes data but must verify Root proof and respect authorization. Service Agents execute business logic but should verify callers and provenance. This role separation reduces implicit trust and creates clearer audit boundaries.

E. The Reference System Should Expose Its Bottlenecks

The prototype is not presented as final Internet-scale infrastructure. It is a working reference that reveals which parts of the lifecycle, publication path, synchronization path, and query path need hardening. OAN treats evaluation as both validation and design feedback.

6. System Roles

OAN separates infrastructure roles from discoverable resource products. Each infrastructure role can be operated by one organization in a small pilot or by different organizations in a larger trust domain. Discoverable products can be published by Agent developers, enterprises, research groups, tool providers, or community maintainers.

A. Root Node

The Root Node is the policy enforcement, verification, and credential issuance center of one OAN trust domain. It observes infrastructure lifecycle state from the on-chain governance layer, issues Root-signed infrastructure authorization VCs, verifies complete resource identity submissions, anchors accepted versions, manages authorization-domain governance, publishes Root-signed records, and creates verified packages for distribution. Root is not expected to handle ordinary business traffic. Its purpose is to establish which resource identities are admissible and to enforce governance-derived infrastructure authorization in protocol interactions.

Root has three hub functions in the trust domain. It is the *trust authorization hub* because it accepts resource versions, issues infrastructure credentials, and signs protocol evidence. It is the *data-distribution hub* because it turns accepted identity state into Root-verified packages and publication cursors consumed by distribution and Discovery services. It is also the *semantic-governance hub* because it governs the ecosystem semantic tag tree used by Registrars during label selection and recommendation and by Discovery nodes during semantic discovery. Root does not perform every search query, but it governs the shared semantic vocabulary that makes cross-node Discovery comparable.

Root maintains the following categories of state:

- active and inactive Registrar authorization state;
- active and inactive Discovery authorization state;
- future third-party VC issuer authorization state;
- Root-issued infrastructure authorization VCs;
- accepted resource identity versions;
- rejected submissions and verification reasons;
- authorization-domain governance metadata;
- semantic tag tree versions and governance metadata;
- publication cursors, package metadata, and bulletin facts.

Root is intentionally conservative. It should reject malformed `did:oan` DID documents, invalid registration credentials, unauthorized Registrar submissions, stale or tampered representations, and authorization-domain inconsistencies.

In the upgraded governance design, Root does not treat the on-chain governance layer as a credential. The governance layer records lifecycle state, while the Root-issued VC is the service credential used in protocol interactions. Effective infrastructure authorization requires both: active governance state and a valid Root-issued VC. This keeps Root as the verification and credential issuer without making Root the only status-query hub for every infrastructure node.

B. Registrar Node

The Registrar Node is the onboarding gateway. It helps resource providers prepare identity drafts, select capability tags, prove control of DID verification methods, receive registration credentials, and submit complete identity representations to Root. Registrar improves usability while preserving Root as the final resource-acceptance and package-verification authority inside the trust domain.

Registrar is not a mere form submission service. It binds operator workflow to protocol evidence. In the reference design, a Registrar draft can move through creation, product-type declaration, semantic description, capability tagging, DID-control challenge, credential issuance, and Root submission. The credential issued by Registrar allows Root to verify that the submitted resource identity passed the expected onboarding path.

C. Discovery Node

The Discovery Node is a controlled query service. It synchronizes Root-verified packages, verifies Root proof, checks bulletin facts, applies authorized-domain authorization, builds local semantic indexes, and returns signed discovery responses. It does not decide that a resource identity is accepted; it relies on Root proof. It also does not expose every package it can fetch. Its own authorization state limits which domains it may index and expose.

Discovery may add ranking, search optimization, local indexes, reputation signals, or domain-specific filters, but these should operate after basic OAN admissibility checks.

For semantic discovery, OAN can draw on the GRAIL line of work, which studies deep-granularity hybrid resonance and SLM-enhanced indexing for real-time Agent discovery [14]. In OAN, such semantic matching is treated as a Discovery-node capability: it improves recall, ranking, and intent matching over verified resource metadata, but it does not replace Root acceptance, package proof, or Discovery authorization checks.

D. CDN Service

The CDN Service distributes Root-verified packages, manifests, metadata, and documents. It improves availability and reduces load on Root. It is not a trust authority. A Discovery node or relying party should verify hashes, signatures, package metadata, and Root proof rather than trusting the CDN path alone.

E. Discoverable Resource Products

OAN supports several resource products under one identity and discovery model:

- **Agent Service**: a network-accessible Agent that exposes callable capabilities and may support A2A, MCP-adjacent, ANP-like, or domain APIs.
- **Skill**: a portable capability description or skill artifact whose DID Document provides semantic discovery fields and may reference the external artifact location.
- **MCP Server**: a tool and resource server whose endpoint and protocol metadata can be bound to a governed `did:oan` identity before MCP initialization.
- **Tool/API**: a conventional API or tool endpoint that benefits from OAN registration, discovery, and credential-bound trust verification.

The important unification is that all these products can be represented by a `did:oan` DID Document with product type, semantic labels, descriptions, use cases, endpoints or artifact references, verification methods, and version metadata. OAN does not need to embed every product-native file in the DID Document. A Skill file, OpenAPI description, or MCP descriptor may live outside OAN and be referenced by URI, while Discovery indexes the DID Document and Root-verified metadata.

This resource model is intentionally narrower than a universal knowledge graph and stronger than a simple endpoint registry. It does not try to describe every domain ontology, pricing rule, runtime permission, or business workflow. It standardizes the minimum trust envelope that different resources need before they can be safely exposed across organizational boundaries: identity control, resource type, endpoint or artifact reference, semantic description, lifecycle state, governance scope, package provenance, and verification material. Application communities can still define richer schemas above this envelope.

The model also avoids tying OAN to one artifact format. An Agent Service may remain a live network service. A Skill may remain a portable package. An MCP Server may keep its native protocol descriptor. A Tool/API may keep an OpenAPI or domain-specific description. OAN does not erase those native forms. It gives them a common identity, publication, and Discovery layer so relying parties can ask the same pre-connection questions before they invoke different kinds of resources.

This makes the resource model suitable for ecosystem growth. New product communities can keep their native runtime or package format while exposing the minimum OAN-facing facts needed for trust: who controls the resource identity, what type of resource it is, which domain authorizes its exposure, which version is current, and which Root-verified package should be used for Discovery and verification. OAN can therefore serve as a shared trust envelope around diverse Agent resources instead of forcing every resource into one marketplace schema or one invocation protocol.

F. Service Agent

The Service Agent is a real business Agent that exposes callable capabilities. It may provide domain knowledge, data access, workflow execution, tool access, or task collaboration. In OAN, a Service Agent has an identity document and can be registered, discovered, and invoked under trust controls.

The Service Agent should verify incoming trusted invocation envelopes. It should check caller DID material, credentials, timestamp freshness, nonce uniqueness, target DID binding, request body hash, and signature. These checks allow the Service Agent to reject replay, substitution, wrong-target, and unauthenticated requests before business logic runs.

G. User Agent

The User Agent queries Discovery, verifies signed discovery responses, selects candidates, checks Root-verified package provenance, builds a signed invocation request, and verifies signed Service Agent responses. The User Agent represents the relying side of OAN. It turns Discovery results into verifiable interaction decisions.

7. OAN Trust Architecture

OAN's trust architecture can be read as an evidence chain. The resource holder first proves control over a `did:oan` identity. A Registrar adds registration evidence after review and subject-control verification. Root checks the submitted identity, Registrar authorization, lifecycle state, and policy constraints before accepting a version. The distribution layer

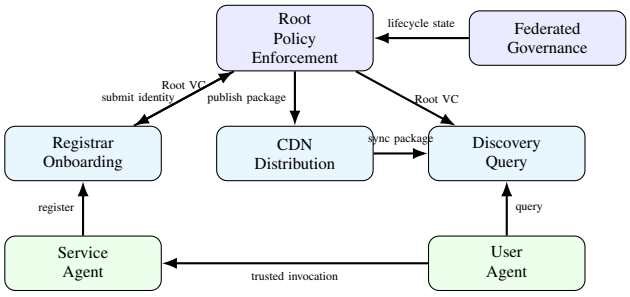


Fig. 2. OAN separates governance state, Root-issued authorization VCs, onboarding, distribution, discovery, and resource interaction.

carries Root-verified packages, while Discovery verifies those packages and applies its own authorized-domain scope before returning signed results. Finally, a relying User Agent and a Service Agent use signed invocation envelopes to bind caller, target, request body, timestamp, nonce, and proof.

This chain is important because no single user-interface element or directory entry is enough to establish trust. Each stage adds a different kind of evidence: subject control, Registrar review, Root acceptance, package integrity, Discovery authorization, response provenance, and invocation freshness. If one stage is missing or stale, a relying party should be able to detect that the candidate is not fully supported by the OAN trust path.

A. Identity Representation

An OAN resource identity representation is based on `did:oan` and is compatible with DID-style concepts. It contains an identifier, verification methods, product type, service endpoints or artifact references, semantic description, capability metadata, credential references, OAN metadata, and version-related information. It also carries typed `authorizedDomains` when the represented subject or resource needs an explicit governance scope. The representation is intended to be complete enough for Root verification, semantic Discovery, and runtime validation. It is not merely a profile card.

The `did:oan` identifier itself is routing-aware. Its method-specific identifier contains a routing component and a uniqueness component. The routing component helps Registrars, Root, and infrastructure nodes check that a DID is being admitted through the expected authority path, but it is not a substitute for proof, VC status, or governance state. This keeps the identifier useful for operational routing while preserving the security rule that trust must come from signatures, credentials, Root acceptance, and current authorization state.

`did:oan` is deliberately resource-oriented. It can describe Agent Services, Skills, MCP Servers, and Tool/API resources without changing the basic trust path. The holder behind the resource still controls private keys and credentials. For callable resources, the holder can use those keys during mutual verification before invocation. For downloadable or descriptor-like resources, the DID Document can provide semantic discovery fields and external artifact references while keeping artifact distribution outside the core OAN protocol.

The identity representation supports three needs at once. First, it provides human-readable and machine-readable metadata for discovery and integration. Second, it provides cryptographic verification material. Third, it gives Root and Discovery enough information to apply lifecycle and authorization checks. In the current design, authorization domains are deliberately separated from capability tags. A tag may help describe or rank a resource, while `authorizedDomains` is the field that Root, Registrar, Discovery, and governance tooling use for scope enforcement.

The same identity model also supports external identifier compatibility. A DID Document may declare external identifiers from existing platforms or industry identifier systems. The external identifier's native `id` remains a metadata value under its own resolution context; it does not replace the OAN DID and does not grant controller authority. This allows OAN to connect existing resource catalogs to a verifiable OAN trust path without forcing every external platform to change its native identifier scheme.

Controller relationships are similarly explicit. A self-controlled resource uses its own OAN Identity to prove control. A dedicated controller is itself an OAN-registered identity that may authorize registrations for other resources. When a resource points to an external controller, the controller value alone is only a declaration; the registration path must verify a controller authorization proof over the resource, controller, Registrar, document hashes, purpose, validity window, and nonce. This prevents a publisher from claiming an arbitrary controller DID without showing control of that controller's key.

B. Registration Credential

The registration credential links Registrar review and resource identity. It binds the Registrar DID, resource DID, credential type, subject-control evidence, validity, product type, and proof. Root verifies this credential before accepting a submitted identity representation. This creates a separation between review workflow and Root acceptance while preserving verifiability.

C. Root Acceptance

Root acceptance is the point where a resource identity version becomes authoritative within a trust domain. Root verifies the submitted representation, credential, signed upstream request, Registrar authorization, DID structure, hashes, capability metadata, and policy constraints. If accepted, Root records the version and publishes verifiable evidence. If rejected, the reason should be auditable.

D. Root-Verified Package

A Root-verified package is the distribution artifact consumed by CDN and Discovery. It contains the identity document or references, hashes, metadata, status, publication cursor, Root proof, and capability information. Discovery does

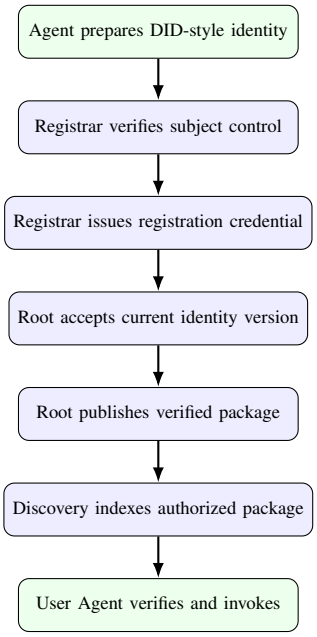


Fig. 3. OAN trust flow from identity preparation to pre-connection invocation.

not trust a package because it came from a CDN; it verifies the package against Root proof and bulletin facts.

E. Signed Discovery Response

Discovery responses are signed. A relying User Agent can verify that a response came from the Discovery node that produced it, that the response is fresh, and that returned candidates carry provenance sufficient for later validation.

F. Identity Directory for Application Login

Some applications need a lightweight way to let a controller or resource use an OAN Identity for login. OAN supports this by allowing applications to query a Root-accepted public-key projection for accepted controller and resource identities. This projection is not a new authorization authority. It helps an application find the public key associated with an accepted DID, after which the application must still verify a fresh challenge signature from the user's OAN Identity. The directory does not replace DID resolution, VC status checks, controller authorization proof, or Root package verification.

G. Trusted Invocation

Trusted invocation extends OAN from discovery into the first step of interaction. A User Agent signs a request envelope that binds caller DID, target DID, timestamp, nonce, body hash, method, path, and proof. The Service Agent verifies the envelope before executing business logic. OAN does not dictate the complete business protocol after this point, but it establishes a trustworthy entry condition.

8. Authorization Domains and Controlled Discovery

Authorization domains are the governance bridge between identity metadata and discovery exposure. A resource may describe many capabilities, but it should only be accepted by a Registrar whose authorized scope covers the resource's `authorizedDomains`; after Root acceptance, it should only be delivered to Discovery nodes whose authorized scope covers those same domains. This is essential for multi-operator deployment. It allows a trust domain to operate domain-specific Registrars and Discovery services without giving each service global exposure power.

Authorization-domain governance has three layers:

- **canonical domains**: a governed domain taxonomy that provides stable top-level and domain-specific categories;
- **infrastructure authorization domains**: the domain set assigned to each Registrar and Discovery node through governance and Root-issued infrastructure authorization VCs;
- **resource authorization domains**: the `authorizedDomains` field bound into the resource DID Document, metadata, Root package, and Registrar-issued resource credential;
- **local refinement tags**: custom tags, descriptions, indexes, or ranking signals used after authorization checks.

The key principle is that custom tags cannot expand governance authority. A Registrar may suggest tags to improve description quality, and a Discovery node may use custom metadata to improve search quality inside an authorized domain, but neither tags nor semantic search may create authorization outside `authorizedDomains`. The special value `["*"]` denotes explicit all-domain authorization, while an empty list denotes no domain authorization. A parent domain covers its descendants by hierarchy, so excluding a child domain also requires excluding any parent that would cover it.

This separation is important for semantic search. Embeddings, small-language model indexes, domain ontologies, and hybrid matching algorithms can improve how a user requirement is mapped to resource candidates, as explored by GRAIL [14]. However, they are ranking and retrieval mechanisms inside an already authorized result space. They should not expand a Discovery node's governed exposure scope.

9. Relationship with MCP, A2A, and ANP

OAN is most useful when understood as a layer around existing protocols rather than a competitor to them.

A. MCP

The Model Context Protocol focuses on connecting AI applications with tools and data sources [7]. It gives clients a structured way to discover and call tools provided by MCP servers. OAN can complement MCP by providing identity and trust context before a tool server or client is used. For example, an Agent can use OAN to verify that an MCP server endpoint belongs to a Root-accepted identity, that the endpoint is current, and that the peer identity has appropriate credentials. MCP remains responsible for tool protocol details.

TABLE I
OAN COMPARED WITH ADJACENT PROTOCOL LAYERS

Layer	Main concern	OAN relationship
MCP	tool and data access	adds identity trust
A2A	Agent task exchange	adds discovery trust
ANP	Agent networking	compatible identity layer
DID/VC	identifier and credential substrate	reused by OAN
Domain APIs	business semantics	can run above OAN

TABLE II
POSITIONING OF OAN AGAINST ADJACENT SYSTEMS

System family	Primary object	OAN relationship
MCP registry	MCP server/tool endpoint	resource type and trust wrapper
Skill marketplace	Skill artifact	governed Skill identity
Agent schema	Agentic schema	compatible metadata schema
ADS	Agent metadata directory	trust-first registry and discovery
ANS	Agent name	complementary alias/name layer
Harness/runtime	task execution	outside OAN data plane

B. A2A

Agent-to-Agent protocols focus on task exchange, capability description, collaboration state, and message flow between Agents [8]. OAN can provide trusted identity onboarding, discovery, and pre-connection validation for A2A-compatible Agents. In a layered deployment, an Agent might first use OAN to verify a discovered peer and then open an A2A session for task negotiation.

C. ANP and Agent DID Methods

ANP-like work and Agent-oriented DID methods explore identity, networking, and interoperability for Agents [9]. OAN is aligned with DID/VC-style identity and can interoperate with Agent DID methods. Its distinctive focus is the complete governed lifecycle: Registrar authorization, Root acceptance, package publication, capability-scoped Discovery, and signed invocation.

D. Layering Summary

The relationship can be expressed as follows: OAN answers whether a resource identity is admissible, current, and discoverable under policy; MCP answers how tools are exposed and called; A2A answers how Agents exchange tasks; ANP and DID methods can provide identity and networking conventions; domain protocols answer how specific industries structure their data and workflows.

OAN's southbound compatibility posture follows from this table. MCP, A2A, ANP, REST APIs, and domain-specific protocols can all remain responsible for their own wire behavior. OAN should add only the pre-connection trust requirements: a resolvable resource DID, Root-verified package evidence, Discovery proof, and credential or signature checks appropriate to the resource type. This keeps OAN broad enough to cover many products without turning it into a universal runtime.

E. Relationship with Registries, Directories, and Schemas

The Agent ecosystem is also developing registries, directories, and schema frameworks that are closer to OAN's problem space than invocation protocols. OAN should be understood as complementary to these efforts, but with a different primary object and trust boundary.

MCP registry work is primarily concerned with making MCP servers and their metadata discoverable [15]. OAN can represent an MCP Server as a resource type and bind its endpoint to a `did:oan` identity, Root acceptance, and verifiable package. In this sense, an MCP registry can be carried by OAN, while OAN's trust path is broader than MCP server lookup.

Skill marketplaces focus on packaging, browsing, and distributing Skills. OAN does not need to replace such marketplaces. Its contribution is to make a Skill a first-class governed resource: a Skill can have a DID, semantic metadata, version state, package proof, and discovery evidence even if the Skill artifact itself is hosted by another marketplace or repository.

Agent schema frameworks aim at common languages for Agentic records, capabilities, Skills, and relationships [16]. OAN's authorization-domain taxonomy and semantic resource fields solve a neighboring problem: how accepted resources are classified and discovered under governance. A future OAN deployment can embed or reference compatible schema records inside a ResourcePackage instead of requiring a separate OAN-only schema vocabulary.

ADS is closer to an Agent directory and discovery fabric, including directory, metadata, routing, and content-distribution concerns [17]. The difference is emphasis. ADS is directory-first: it starts from scalable discovery of Agent metadata. OAN is trust-first: it starts from governed registration, Root acceptance, verifiable distribution, authorization-aware Discovery, and signed use of results. The two approaches can converge when ADS or external schema records are treated as resource metadata inside OAN's trust envelope.

ANS and related Agent naming systems focus on name assignment and name-based discovery for Agents [18], [19]. OAN is DID-first and resource-first. It can interoperate with names as aliases or service metadata, but its core object is the trusted resource identity and ResourcePackage rather than the name alone.

This layering is the core positioning of OAN. It lets a partner keep an MCP server, an A2A-capable Agent, an ANP-like route structure, or a domain API while adding a common trust path before discovery and invocation. OAN should therefore be evaluated not as a rival to those protocols, but as the layer that can make cross-protocol Agent discovery and first contact governable.

10. OPERATIONAL DEPLOYMENT PATTERNS

A. Single-Operator Trial Network

The simplest deployment places Root, Registrar, Discovery, CDN, Service Agent, and User Agent under one operator. This pattern is useful for development, testing, demonstration, and

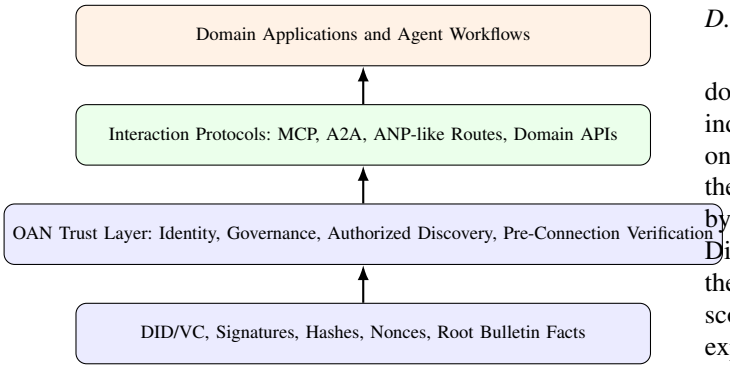


Fig. 4. OAN is positioned as a trust layer below Agent interaction protocols and above cryptographic primitives.

early governance experiments. It validates lifecycle correctness without introducing cross-organization operations too early.

B. Federated Infrastructure Governance

In a multi-organization deployment, governance should not be reduced to the private decision of one Root operator. OAN therefore separates committee-level infrastructure lifecycle decisions from Root-side protocol enforcement. A governance committee can authorize, suspend, recover, or revoke Registrar, Discovery, and future VC issuer nodes through threshold-governed actions published by the governance layer. Root observes these decisions and issues or refuses infrastructure authorization VCs accordingly.

This pattern changes the system from a single-operator trust path into a federated trust-governance architecture. Root still anchors accepted resource versions and signs packages, but it does not need to be the sole institution that controls which infrastructure participants may exist. That distinction is important for standards-facing deployment, because partner organizations can participate in governance without forcing ordinary Agents or resource providers to understand governance operation code.

C. Multi-Registrar Onboarding

As the ecosystem grows, multiple Registrars can onboard Agents for different communities or domains. Root remains the final resource-acceptance and package-signing authority inside the trust domain, while Registrar lifecycle authorization is governed above Root. This pattern allows domain operators to perform onboarding work while keeping a consistent trust record. Registrar authorization is domain-scoped. A Registrar may accept a resource only when the resource's `authorizedDomains` are covered by the Registrar's own authorized domain set. This prevents a domain-specific Registrar from becoming a global admission gateway. When a resource does not explicitly provide domains, an implementation may reject the submission, default it to the Registrar's approved scope, or derive a reviewed subset, depending on local onboarding policy; in every case the final domains should be explicit in the accepted representation.

D. Domain-Specific Discovery

Different Discovery nodes can serve different authorized domains. For example, one Discovery node may focus on industrial Agents, another on research Agents, and another on public-service Agents. Each Discovery node receives only the authorization it needs. Root targets Discovery notification by intersecting resource `authorizedDomains` with each Discovery node's authorized domains. Discovery then verifies the Root package and refuses to index packages outside its scope. Query matching therefore runs after governance-bound exposure, not before it.

E. Partner Integration

A partner organization can keep its existing Agent runtime and protocol stack while adding OAN registration, discovery, and verification. It may expose an OAN-compatible identity document, register through a Registrar, synchronize with an authorized Discovery node, and add verification before MCP or A2A interaction.

F. Production Deployment

A production deployment should add database hardening, key management, observability, rate limiting, replay protection storage, backup, monitoring, deployment automation, and conformance tests. OAN's current reference implementation provides the architecture and protocol path; production operation requires ordinary platform engineering around it.

11. Governance Model

OAN governance is divided between committee-level infrastructure decisions, Root-side policy enforcement, and protocol evidence. Policy decisions include who may operate a Registrar, which domains a Registrar may admit, which domains a Discovery node may expose, how authorization domains are maintained, what evidence is required for registration, and how revocation is handled. Protocol evidence turns these decisions into signed, verifiable facts.

A. Infrastructure Authorization

Infrastructure authorization has two linked layers. The on-chain governance layer records lifecycle state for Registrar, Discovery, and future third-party VC issuer nodes. Root then issues infrastructure authorization VCs to nodes whose governance state is active and whose DID-control and operational materials pass Root verification.

Neither layer is sufficient alone. A Registrar with a valid Root VC should not be accepted after its governance state becomes inactive. A Discovery node whose governance state is active should not be treated as protocol-authorized before Root has issued the corresponding VC. Effective authorization is therefore:

$$\textit{active governance state} \wedge \textit{valid Root-issued VC}.$$

This distinction lets the governance layer provide auditable lifecycle changes while Root remains the issuer and verifier

of protocol credentials. In this sense, OAN is closer to a federated trust-governance infrastructure than to a plain Agent registry. The governance layer answers who is allowed to operate infrastructure; Root enforces that answer and signs the protocol artifacts used by services; Discovery exposes only Root-accepted and policy-admissible resource identities.

B. On-Chain Governance Layer

For infrastructure participants, OAN can publish lifecycle authorization changes through an on-chain governance layer. The layer records and emits events for Registrar, Discovery, and future third-party VC issuer lifecycle changes, including authorization, suspension, recovery, revocation, and Registrar/Discovery authorized-domain updates. Infrastructure services consume the event stream through an indexer or local cache, rather than asking Root for current status on every request.

This layer is not a storage layer for resource identities. It does not store, parse, validate, or issue DID Documents, Verifiable Credentials, metadata, or capability descriptions. These remain OAN protocol and service responsibilities. The governance layer may carry external references such as subject identifiers and hashes of review material, but its core function is participant lifecycle management.

This boundary is deliberate. Governance events are low-frequency and high-value: node authorization, suspension, recovery, revocation, and authorized-domain changes should be auditable and hard to rewrite. Resource registration, package synchronization, semantic indexing, and Discovery queries are higher-frequency runtime activities and should remain off chain. OAN uses the chain as an authorization and audit substrate, while Root, Registrar, Discovery, and indexer services translate those facts into efficient runtime behavior.

The governance operation model keeps proposal creation, review, voting, and lifecycle actions in designated governance clients or processes, separate from ordinary Registrar, Discovery, and VC issuer service runtimes. The intended direction is federated committee operation with threshold approval, so the system can move from initial stewardship toward broader institutional participation. Registrar, Discovery, and VC issuer service nodes are event consumers by default. As the ecosystem matures, governance operation clients can be opened to more participating organizations without changing ordinary resource-provider or service-node integration.

C. Resource Admission

Resource admission involves both operator workflow and protocol checks. A Registrar may help review metadata and verify subject control. Root then verifies the complete representation and supporting evidence. Admission should not depend only on a website account, email address, or human-readable description. The same admission path can handle Agent Service, Skill, MCP Server, and Tool/API products because their OAN-facing representation is a `did:oan` DID Document plus verifiable registration evidence.

D. Versioning and Revocation

Resources change. Endpoints move, keys rotate, capability descriptions evolve, Skills gain new releases, APIs add or remove operations, and operators may be revoked. OAN treats identity as versioned state. A current version should supersede older accepted versions, and revoked or inactive versions should not be discoverable after synchronization.

E. Auditability

Operators should be able to answer who submitted an identity, which Registrar issued the credential, which Root decision accepted or rejected it, which package was published, which Discovery node indexed it, and why a request was accepted or rejected. This audit path is central to cross-organization trust.

12. Security and Risk Posture

OAN is not a universal security solution. It is closer to zero-trust and workload-identity thinking than to a model-safety framework [20], [21]. It does not guarantee that an Agent's business behavior is correct, truthful, safe, or high quality. It addresses the infrastructure trust questions around identity, governance, discovery, and pre-connection verification.

A. Threats Addressed

The architecture is designed to reduce several risks:

- identity imitation through unsigned or unaccepted metadata;
- controller impersonation through unsupported controller strings;
- stale endpoint use after identity update or revocation;
- unauthorized Registrar submission;
- Discovery exposure outside authorized domains;
- tampering with distributed packages;
- replay or wrong-target invocation requests;
- blind trust in directory output without provenance verification.

B. Threats Not Fully Addressed

OAN does not by itself solve prompt injection, model hallucination, malicious tool behavior, data leakage inside business logic, semantic misrepresentation of capabilities, or economic reputation. These require additional application, model, policy, and operational controls. OAN makes it easier to bind those controls to verifiable identities, but it does not replace them.

13. Prototype and Evaluation Status

The current OAN prototype has been evaluated through lifecycle correctness, negative verification, authorization-aware Discovery, internal degraded baselines, and scalability characterization. The reported single-node experiments cover 10, 50, 100, 200, 500, 1000, and 2000 identity representations. Logical multi-node experiments cover three Registrar nodes, two Discovery nodes, one Root node, and one CDN node, with per-Registrar scales from 10 to 400, reaching 1200 total identity representations.

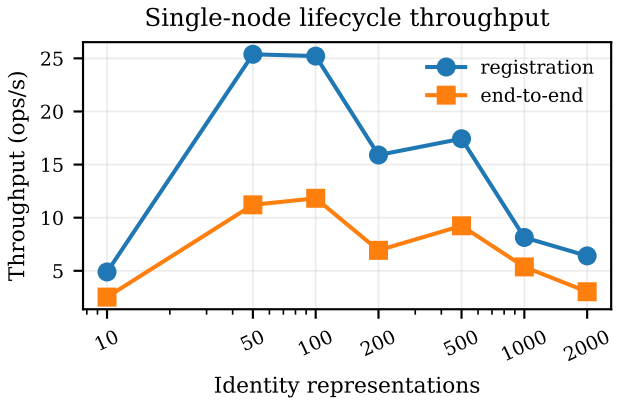


Fig. 5. Single-node lifecycle throughput across increasing identity scales. Registration throughput and end-to-end throughput remain measurable through 2000 identity representations, while the larger scales reveal where production queueing and indexing work should focus.

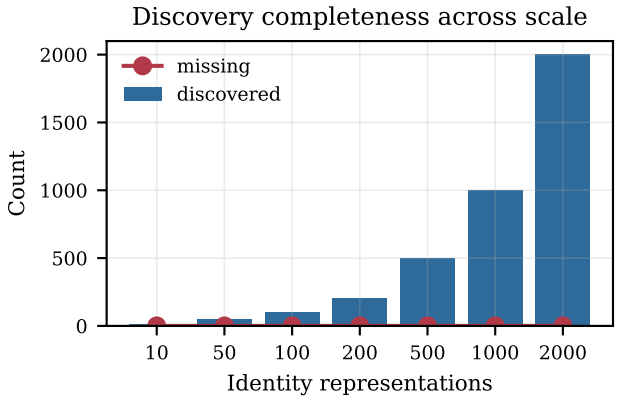


Fig. 6. Discovery completeness in the single-node scalability experiment. Every tested identity representation was discoverable after synchronization, with zero missing results across scales from 10 to 2000.

The prototype remained operational in the tested scales and returned complete authorized Discovery results in the reported runs. The main bottlenecks appear in large-result query processing, publication, synchronization, and batch submission under high load. These results should be interpreted as feasibility and bottleneck evidence, not as a final production-capacity claim. They also should not be read as a complete production benchmark suite. A production deployment still needs independent measurements for high availability, geographically distributed nodes, long-running synchronization, large tenant populations, and operator-specific storage and queue choices. The public reference deployment has also exercised a larger resource corpus and multi-node registration and discovery paths. Those observations are useful for architecture validation, especially around distribution tails and Discovery indexing costs, but they should be treated as operational experience rather than controlled benchmark results.

Figures 5–7 summarize the main empirical evidence for the white paper. The results support three practical claims: the

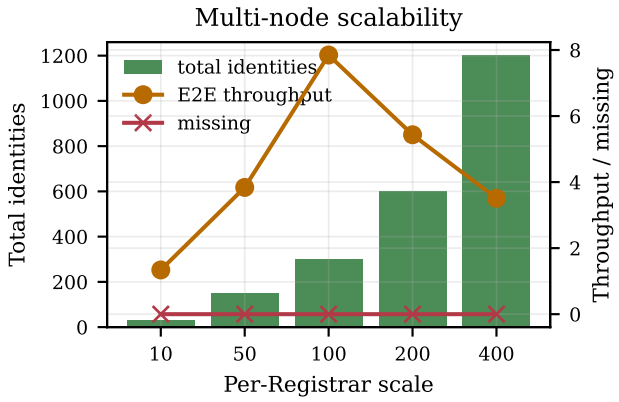


Fig. 7. Logical multi-node scalability with three Registrar nodes and two Discovery nodes. The largest run accepted 1200 total identity representations and both Discovery nodes returned the complete authorized result set.

TABLE III
REPRESENTATIVE EVALUATION COVERAGE

Experiment type	Purpose	Coverage
Lifecycle	verify normal path	registration to query
Negative cases	reject invalid inputs	credentials, proof, scope
Discovery auth	test domain filtering	in-scope and out-of-scope
Ablation	isolate mechanisms	degraded internal baselines
Scalability	find bottlenecks	up to 2000 identities
Multi-node	test role relations	3 Registrar, 2 Discovery

lifecycle can be executed end to end, authorized Discovery remains complete in the reported runs, and the role-separated design can operate across multiple logical Registrars and Discovery nodes. The same results also identify engineering work for future deployments, especially pagination for large result sets, batch tuning, and stronger production storage backends. In the reference architecture, structured resource visibility and semantic ranking should also be interpreted separately: a Discovery node may accept and expose Root-verified structured resource data before a derived semantic index has fully caught up.

14. How to Interpret the Prototype Results

The current prototype results are important because they validate the coupling among lifecycle governance, Discovery authorization, and trusted invocation. They should be read with the correct expectations. OAN is not claiming that a single reference deployment has already reached global Internet scale. It is claiming that the core trust workflow can be executed end to end, that invalid or unauthorized inputs can be rejected in controlled experiments, and that the main performance bottlenecks are visible enough to guide hardening.

A. Lifecycle Correctness

Lifecycle correctness means that the same resource identity representation moves through preparation, credentialing, Root acceptance, package publication, Discovery synchronization, query, and invocation without losing the identity binding. This

is the most fundamental requirement. A fast system that cannot preserve this binding is not useful as trust infrastructure.

B. Negative Verification

Negative verification demonstrates that the system rejects malformed or unauthorized inputs. These tests are as important as successful-path tests because trust infrastructure is judged by what it refuses to accept. Invalid credentials, wrong subjects, unauthorized infrastructure identities, stale versions, and out-of-scope Discovery cases should fail deterministically.

C. Authorization-Aware Discovery

Authorization-aware Discovery demonstrates that query matching is not enough. The returned candidates must also satisfy Root acceptance, current-version state, and authorized-domain authorization. This result supports OAN's central claim that Discovery in an Agent Internet should be a governed exposure function, not only a search box.

D. Scalability Characterization

The scalability results identify where engineering effort should go next. Large query result sets, publication batching, synchronization, and Registrar submission pressure are natural bottlenecks. These observations provide an engineering basis for prioritizing pagination, indexing, queue tuning, backpressure, and database backend evolution.

15. Public Pilot Network and Runtime Transparency

The reference architecture has moved beyond a local prototype into a public pilot network. This status should be interpreted carefully. The pilot is not a claim of final Internet-scale production capacity, and it is not a new performance benchmark. Its value is methodological: it makes the trust path observable through a running multi-node setting, public documentation, and reference interaction surfaces that external readers can inspect.

These public surfaces do not become protocol authorities. They help readers observe how Registrar, Root, Discovery, and governance-backed trust evidence fit together, while the authoritative trust decisions remain in the protocol objects, signatures, governance state, Root acceptance, and Discovery verification path.

A. Multi-Node Pilot Profile

The current public pilot demonstrates multiple Registrar and Discovery nodes under one governed trust domain. The architectural point is not the current node count, but the fact that multiple nodes can follow the same admission, authorization-domain, distribution, and Discovery rules. Additional Registrar nodes can accept resources within their authorized domains; additional Discovery nodes can synchronize accepted packages and expose resources within their authorized domains. This allows OAN to grow by adding operators without changing the meaning of registration, Root acceptance, package publication, or Discovery authorization.

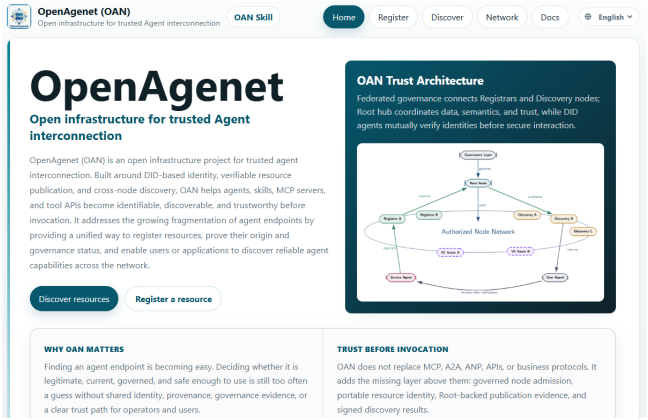


Fig. 8. Public interaction surface for OAN. The surface illustrates how registration, discovery, network transparency, and documentation can be exposed without becoming independent trust authorities.

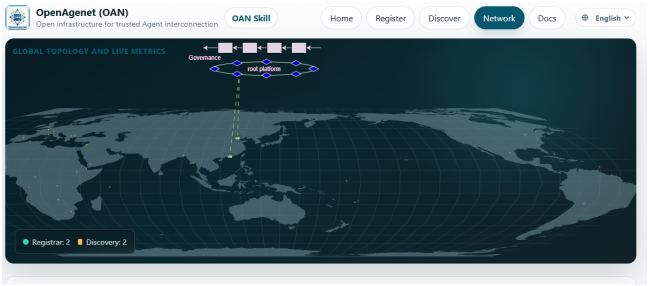


Fig. 9. Public Network topology view. The view presents a human-readable snapshot of OAN node participation and topology without exposing internal deployment details.

This multi-node profile also clarifies an operational requirement: a newly authorized Discovery node should not only receive future updates. It should be able to backfill existing authorized resources from Root or the distribution layer and converge toward the same visible resource set within its scope. That requirement belongs to production hardening and interoperability work, not to a new experimental claim.

B. Network Transparency

Network transparency is a public observability principle, not an operator dashboard requirement. A deployment may expose topology, resource counts, resource-type distribution, recent registration activity, Discovery activity, node-level participation, and a curated history of governance events. These signals help readers understand that OAN is a governed network with observable state, not a static diagram.

Network transparency must remain separate from protocol authority. Public statistics can describe resource counts, resource categories, node-level activity, and governance-event summaries. They should not expose private logs, operator secrets, internal service details, or business payloads. They also should not be treated as governance state or as formal performance evidence. The authoritative trust path remains the combination of governance state, Root acceptance, Root-

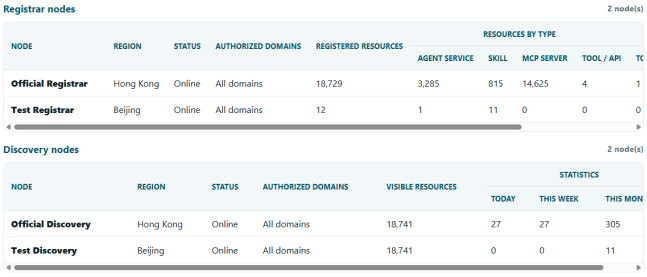


Fig. 10. Node-level public metrics. Public metrics make Registrar and Discovery participation visible while keeping protocol trust and operator secrets outside public presentation surfaces.

issued evidence, package verification, and Discovery response verification.

16. Standardization Considerations

OAN is not yet a formal standard, but it is designed to produce standardizable surfaces. A standardization path should be careful: it should standardize stable trust semantics before freezing every implementation detail.

A. Candidate Standard Surfaces

Candidate surfaces include Agent identity representation profiles, registration credential claims, signed upstream envelopes, Root-verified package metadata, Discovery response proof format, trusted invocation envelope, authorized-domain authorization semantics, and conformance test vectors. These surfaces are useful because they are observable at interoperability boundaries. They should also include the `did:oan` identifier profile, DID Document proof profile, OAN Identity handling conventions for local private material, controller authorization proof format, external identifier representation, and the public-key discovery semantics needed by applications that verify controller or resource login signatures.

OAN's standardization surface should also be understood in relation to wider Agent identity and governance work. National and industry specifications for Agent identity can define important identity-code and identity-management conventions. OAN can complement such work by defining how resource identities are registered, accepted, packaged, discovered, and verified across nodes. Similarly, CCSA-oriented proposals, AIP-style Agent protocols, and other Agent identifier systems can be mapped into OAN when their identity objects, aliases, or governance facts are represented inside OAN-compatible resource metadata.

The governance layer should remain chain-neutral at the architectural level. Public blockchains can carry governance evidence for open ecosystems, while consortium chains can serve regulated or industry-specific deployments. Off-chain indexers can translate those low-frequency governance facts into runtime authorization state consumed by Root and infrastructure nodes. This keeps high-frequency registration and discovery off the governance chain while preserving traceable authorization history.

B. What Should Remain Flexible

Operator review process, user interface, database backend, local indexing strategy, ranking algorithm, business workflow, and domain-specific Agent protocols should remain flexible. Standardizing these too early would reduce adoption. OAN should standardize the trust path and let ecosystem participants innovate above it.

C. Conformance Before Branding

A project should not be considered OAN-compatible merely because it uses the name or exposes similar endpoints. It should pass conformance tests for object canonicalization, signature verification, credential proof, Root package verification, Discovery authorization, current-version handling, and replay rejection. It should also pass DID Document proof tests, OAN Identity generation and import checks, controller authorization proof checks, external identifier handling tests, and public-key discovery checks for login-style verification. This protects the ecosystem from shallow compatibility.

17. Implementation Guidance

A. Start with the Trust Path

New implementers should begin with the trust path rather than UI or ranking. The minimal useful path is: create identity representation, prove DID control, issue registration credential, submit to Root, accept and package, synchronize Discovery, query, verify response, and perform signed invocation. Once this path works, local usability and search quality can improve safely.

B. Keep Business Traffic Out of Root

Root should not become a universal business proxy. It should govern identity, infrastructure authorization, and package publication. Business traffic between Agents should be handled by Agent protocols and application systems, with OAN verification used at connection boundaries.

C. Treat Custom Tags Carefully

Custom tags are useful for search and domain adaptation. They are also risky if they accidentally bypass governance. Implementations should treat canonical authorized domains as the authorization basis and custom tags as refinement only.

D. Design for Failure

Registrars may be unavailable, Root publication may lag, CDN synchronization may fail, Discovery may be stale, and Service Agents may reject invocation envelopes. A production implementation should surface these states clearly. Operators need to know whether a missing candidate is caused by registration failure, Root rejection, publication delay, Discovery authorization, query mismatch, or runtime verification failure.

18. Use Cases

A. Trusted Public-Service Agent Directory

A public-service operator may need to expose Agents for policy consultation, service navigation, document preparation, or process guidance. OAN allows such Agents to be registered, governed, and discovered under a shared trust domain. Users and platforms can verify that a listed Agent is Root-accepted and current.

B. Industrial Agent Cooperation

Industrial Agents may operate across manufacturing, logistics, quality control, energy management, and supply-chain workflows. Authorized-domain Discovery can separate governed exposure scopes while allowing approved cross-domain cooperation. Existing industrial protocols can remain in place while OAN provides identity and discovery trust.

C. Research and Testbed Collaboration

Universities, laboratories, and standards groups can use OAN as a testbed for Agent identity, discovery, governance, and interoperability. The role separation makes it possible to test multiple Registrar and Discovery operators without rewriting the whole stack.

D. Enterprise Agent Gateway

An enterprise can register internal or partner-facing Agents through a Registrar, expose selected capabilities through a Discovery node, and require OAN verification before external Agents access internal workflows. This pattern can coexist with enterprise IAM, API gateways, and data-governance systems.

19. Cooperation with Other Organizations

OAN is useful for cooperation because it does not require partners to abandon their current systems. A partner can keep its own Agent framework, MCP tools, A2A task logic, enterprise workflow engine, or domain API while integrating OAN for identity and discovery.

Practical cooperation can begin with a small set of tasks:

- map the partner's Agent, Skill, MCP Server, or Tool/API resources to OAN resource types and authorized domains;
- create OAN-compatible `did:oan` documents for selected resources;
- register selected resources through a trial Registrar;
- expose them through an authorized Discovery node;
- add OAN verification before MCP, A2A, REST, or domain-protocol sessions;
- define conformance checks and shared test cases.

This lets partners collaborate on the trust layer first, without forcing early agreement on every application protocol detail.

For organizations that already use ADS, ANS, MCP registries, Skill marketplaces, the preferred cooperation model is not replacement. It is wrapping and mapping. Existing records can be mapped into OAN ResourcePackages; existing names can be represented as aliases; existing schema objects can be embedded or referenced as metadata; and existing runtimes can keep executing tasks after OAN has completed pre-connection verification.

The open-source path is also part of cooperation. A partner should be able to start from documentation, inspect the protocol and reference services, use SDKs for Agent-side verification, register a small number of resources, and then decide whether it needs to operate infrastructure nodes. This keeps the first step small while preserving a route toward deeper participation. It also makes standardization discussion more concrete: participants can debate object models, trust evidence, conformance checks, and governance vocabulary against a running reference stack instead of only comparing abstract diagrams.

20. Stakeholder Value

OAN has different value for different stakeholders. A useful white paper should make these differences explicit because the system is not only a technical component; it is also an ecosystem coordination mechanism.

A. Resource Developers

Resource developers need a way to publish Agent Services, Skills, MCP Servers, or Tool/API endpoints that can be trusted outside their own application. OAN gives them a path to describe identity, endpoints or artifact references, capabilities, credentials, and verification methods in a format that other systems can inspect. Instead of asking every possible partner to trust a private document or a manually shared endpoint, the developer can point to an accepted OAN identity and a discoverable package.

For developers, OAN also reduces repeated integration work. Once a resource can produce OAN-compatible identity material and signed responses, it can be made visible through different Discovery nodes and can be invoked by different User Agents. This is especially valuable for small teams that cannot negotiate a custom trust integration with every potential partner.

B. Infrastructure Operators

Infrastructure operators need clear responsibilities. OAN separates Root, Registrar, Discovery, and CDN so that an operator can choose a role without owning the entire ecosystem. A registrar operator can focus on onboarding and evidence collection. A Discovery operator can focus on search, indexing, and domain-specific query quality. A Root operator can focus on governance-state consumption, policy enforcement, audit, credential issuance, and accepted-version authority. A CDN operator can focus on package availability.

This role separation also allows multiple organizations to participate in a pilot network without collapsing into one centralized service provider.

C. Enterprise Users

Enterprise users usually need three assurances before they connect external Agents to internal workflows. First, the external Agent must be identifiable. Second, its discoverability

should be governed. Third, invocation should carry verifiable caller and target context. OAN provides these assurances without forcing the enterprise to expose internal business systems through OAN itself. The enterprise can keep its existing access-control, auditing, and data governance mechanisms while using OAN as a pre-connection trust layer.

D. Public-Service and Industry Platforms

Public-service and industry platforms often require transparent governance and traceability. OAN's Root acceptance records, Registrar credential path, and Discovery authorization model provide a concrete basis for explaining why an Agent appears in a directory. This is important when Agents represent public service functions, regulated-domain guidance, or industrial workflows where uncontrolled exposure can create real operational risk.

E. Standards and Research Communities

OAN provides a concrete implementation target for discussions about Agent identity, Agent discovery, and Agent-to-Agent trust. It is not merely a conceptual diagram. Its multi-repository implementation, tests, examples, and experimental results can support standards discussion, interoperability tests, and academic evaluation.

21. Governance Operations

The OAN governance model should be practical enough for operators. It should not stop at saying that Root is trusted. It should define the kinds of operational decisions that Root and ecosystem maintainers must perform.

A. Registrar Authorization Lifecycle

A Registrar authorization lifecycle includes application, review, activation, monitoring, suspension, and revocation. During application, an operator describes its intended authorized domains, security posture, key management plan, and operational contact. During activation, the governance operator records the Registrar DID, lifecycle state, and approved authorized domains in the bulletin. During monitoring, Root and ecosystem maintainers may inspect submission quality, rejection patterns, abuse reports, or operational health. Suspension and revocation should be reflected in Root state so that future submissions are rejected.

B. Discovery Authorization Lifecycle

Discovery authorization is domain-sensitive. A Discovery node should receive a defined authorized-domain scope. That scope can be expanded, reduced, paused, or revoked. Because Discovery controls exposure, its authorization should be treated as a public trust fact. The on-chain governance layer allows Discovery authorization changes to be sensed by infrastructure nodes through events. A relying Agent should be able to know whether a Discovery node is currently authorized and which domains it may expose through verified Discovery evidence.

C. Semantic Tag Tree Governance

The authorization-domain taxonomy and semantic tag tree are not only technical indexes. They are governance artifacts. Adding, renaming, splitting, or merging domains can affect which resources a Registrar may admit and which resources become visible through which Discovery nodes. Adding or reorganizing semantic tags can affect how resources are described during registration, how tags are recommended to publishers, and how Discovery maps user intent to resource candidates. OAN therefore treats domain-taxonomy and semantic-tag changes as governance operations. A production ecosystem should keep versions, change records, migration guidance, and compatibility rules for authorization domains and semantic tags.

D. Incident Response

An Agent may be compromised, a Registrar may issue invalid credentials, a Discovery node may expose out-of-scope identities, or a package distribution path may be attacked. OAN should support incident response through revocation, state updates, publication of new Root facts, package resynchronization, Discovery re-indexing, and relying-party verification. The goal is not to prevent every incident, but to make incidents observable and recoverable.

22. Data and Evidence Model

OAN separates several kinds of data that are often mixed together in simpler directory systems.

A. Public Identity Data

Public identity data includes resource DID, verification methods, service endpoints or artifact references, capability tags, public metadata, controller declarations, external identifiers, and Root-verified package references. This data is meant to be used for discovery and verification. It should be sufficient for a relying party to decide whether a candidate is a valid OAN identity and how to approach it. Private key material remains outside the public identity data model. An OAN Identity may hold private signing material locally, but a DID Document, VC, Root package, Discovery response, or public identity directory entry should expose only public verification material and verifiable metadata.

B. Controller and External-Identifier Evidence

Controller evidence explains who may authorize a registration operation or control a resource relationship. A self-controlled resource binds its own DID and keys directly. A dedicated controller or external controller uses a separate controller identity and explicit authorization evidence. This separation allows applications to verify a controller signature without treating a controller string as sufficient proof of authority.

External-identifier evidence has a different purpose. It maps an OAN resource to a native platform or industry identifier. Such a mapping improves interoperability and traceability, but it is not by itself proof that the external platform has

endorsed the OAN resource. Relying parties should interpret it as controller-signed metadata unless a separate external-platform proof is also available.

C. Governance Evidence

Governance evidence may include operator applications, review notes, enterprise approval documents, or offline contact verification. This evidence may be important for governance, but it does not always belong in public packages. OAN therefore allows the protocol-facing trust path to rely on signed credentials and Root records while leaving private evidence in controlled operational systems.

D. Runtime Invocation Evidence

Runtime invocation evidence includes signed request envelopes, nonces, timestamps, selected-service provenance, and signed responses. This evidence is used to verify a specific interaction attempt. It is different from identity admission evidence. Keeping the two separate helps avoid requiring Root to observe every business request.

E. Audit Evidence

Audit evidence links the lifecycle together: which Registrar submitted a representation, which credential was used, which Root record accepted it, which package was published, which Discovery node indexed it, and which response was returned. OAN's architecture is designed so that this chain can be reconstructed without relying on informal claims.

23. Interoperability Strategy

OAN's interoperability strategy has four layers.

A. Object Compatibility

The first layer is object compatibility. DID-style documents, credential-like claims, signed envelopes, Root packages, and Discovery responses should have stable schemas and canonical verification rules. SDKs should help developers produce and consume these objects without hand-implementing every cryptographic detail. OAN Identity handling belongs at this layer as a local-key convention: implementations should never confuse local private signing material with public DID Documents, VCs, or Discovery responses.

B. Protocol Adapter Compatibility

The second layer is adapter compatibility. MCP, A2A, ANP-like protocols, REST APIs, and domain protocols can use OAN verification before opening a session or executing a request. The adapter should translate OAN trust context into the local protocol's authentication, authorization, or metadata model. For example, an application login adapter can map an OAN Identity challenge signature and a Root-accepted public-key projection into the application's session model without making the application itself a DID resolver or VC issuer.

TABLE IV
Representative Adoption Depths

Depth	Suitable participants	Typical action
Observe	reviewers, visitors	inspect portal, Network, Docs
Publish	resource providers	register resources
Discover	users, applications	query trusted candidates
Integrate	developers	use SDKs and DID method docs
Operate	node operators	prepare Registrar or Discovery nodes

C. Operational Compatibility

The third layer is operational compatibility. Operators need deployment, configuration, logging, and monitoring conventions that make independent implementations comparable. Without operational compatibility, a protocol can look interoperable on paper while failing in real deployments.

D. Governance Compatibility

The fourth layer is governance compatibility. Different organizations may use different review processes, but they need shared meanings for accepted, revoked, authorized, current, and discoverable states. OAN's governance-layer lifecycle events, Root-issued infrastructure authorization VCs, Root records, and authorization-domain model provide a basis for this shared vocabulary.

24. Adoption Path

OAN adoption can be staged. A staged path lowers integration risk and allows partners to choose a depth of participation. The stages are not mandatory in a fixed order. A partner may begin as an SDK consumer, operate a Discovery node for one capability domain, run a Registrar for a governed community, or build an adapter that brings OAN verification into an existing MCP, A2A, ANP-like, or domain-specific protocol. This flexibility is important for cooperation with organizations that already have operational systems and cannot replace them in one step.

Table IV shows that OAN can be adopted at different depths. A visitor can inspect the public network. A resource provider can register an Agent Service, Skill, MCP Server, or Tool/API endpoint. A developer can use SDKs and the `did:oan` method specification. An infrastructure operator can prepare a Registrar or Discovery node under governance admission. This layered path is important because open infrastructure must support both lightweight evaluation and deeper operational participation.

Public interaction surfaces and SDKs serve different adoption moments. The public surfaces make the network understandable and testable. SDKs let applications generate OAN Identity material, verify DID Documents, verify credentials, consume Discovery responses, and create signed invocation evidence without reimplementing the trust path. These adoption mechanisms should remain consistent with the same underlying trust model rather than becoming separate sources of authority.

A. Stage 1: Identity Publication

In the first stage, an organization creates OAN-compatible identity documents for a small number of Agents. The goal is to make identity material explicit: DID, verification methods, endpoints, capabilities, and basic metadata.

B. Stage 2: Governed Registration

In the second stage, Agents are registered through Registrar and accepted by Root. The organization learns the evidence path, subject-control proof, and Root package publication workflow.

C. Stage 3: Trusted Discovery

In the third stage, an authorized Discovery node indexes accepted packages and returns signed discovery responses. Partner systems can start using OAN Discovery instead of manually curated endpoint lists.

D. Stage 4: Trusted Invocation

In the fourth stage, User Agents and Service Agents use signed invocation envelopes. The integration now covers the full pre-connection path from identity to first verified request.

E. Stage 5: Protocol Integration

In the fifth stage, OAN verification is embedded into MCP, A2A, ANP-like, or domain-specific protocol flows. At this point, OAN becomes part of routine Agent interconnection rather than a separate demonstration.

25. Economic and Ecosystem Considerations

Open infrastructure must avoid unnecessary lock-in. OAN's role separation supports different ecosystem models. A public trust domain may operate one Root with multiple independent Registrars and Discovery nodes. An enterprise may operate a private Root for internal and partner Agents. A research testbed may allow experimental roots and compare policy designs. A standards community may use OAN object models and conformance tests without mandating a single public operator.

The ecosystem should also allow specialization. Some Discovery nodes may optimize for low-latency industrial search. Others may optimize for public service transparency. Some Registrars may specialize in enterprise Agents, while others focus on research or developer communities. OAN gives these roles shared trust semantics while allowing different service models above them.

26. Limitations

OAN's current design has limitations that should be stated clearly.

A. Single Trust Domain Scope

The current architecture describes one governed trust domain at a time. Inside that domain, infrastructure authorization can already be committee-governed above Root through the on-chain governance layer. What remains future work is cross-domain or multi-root federation: how two independently governed OAN trust domains recognize each other's Root records, governance policies, and Discovery exposure rules. Until that federation profile is defined, cross-domain trust requires explicit integration or policy mapping.

B. Semantic Quality

OAN verifies identity and governance state. It does not prove that an Agent is competent, safe in all contexts, or truthful in every response. Domain testing, reputation, certification, runtime monitoring, and policy enforcement remain necessary.

C. Privacy

OAN currently emphasizes verifiable discovery. Some environments need private queries, selective disclosure, or confidential capability matching. These are future extensions rather than solved features.

D. Production Hardening

The reference implementation is suitable for architecture validation and trial integration. Production deployment requires hardening around database backends, key custody, rate limits, monitoring, backup, disaster recovery, and operational security.

27. Reference Implementation Profile

The OAN reference implementation follows the role separation described in the architecture. Root provides acceptance, proof issuance, package publication, distribution coordination, and trust-domain services. Registrar provides governed onboarding, DID Document validation, registration review, and Root submission. Discovery provides package synchronization, structured indexing, derived semantic indexing, scoped query, and signed Discovery results. A governance-indexing component converts low-frequency governance events into runtime-readable infrastructure authorization state. Client libraries help applications create OAN Identity material, produce signed envelopes, verify DID Documents and VCs, and consume Discovery responses.

The important point is not a particular repository layout or deployment procedure. It is the separation of public protocol roles from implementation choices. Partners can adopt only the parts they need: resource publishers may start with identity generation and registration, application developers may use client-side verification, and infrastructure operators may run Registrar or Discovery nodes under governance admission. This lets OAN grow as open infrastructure rather than a monolithic application.

28. Roadmap

A. Short-Term

Near-term work should consolidate the public pilot network and its developer entry points. Registration and discovery interaction surfaces, network transparency, multi-Registrar and multi-Discovery operation, node-level public signals, governance-event summaries, and reference implementation materials should become stable evaluation and onboarding surfaces. Additional work should improve SDK usability, DID and VC conformance vectors, OAN Identity ergonomics, controller-aware registration flows, external-identifier interoperability, operator-facing documentation, and third-party node admission materials.

B. Medium-Term

Medium-term work should improve production readiness: database abstraction, durable persistence, stronger key management, durable nonce storage, pagination, index optimization, observability, rate limiting, and operational governance practices. Adapters for MCP and A2A should move from placeholder compatibility to practical integration patterns. The project should also refine semantic Discovery quality, result inspectability, public metric definitions, asynchronous derived-index recovery, backfill behavior for newly admitted Discovery nodes, and compatibility documentation for national standards, industry proposals, AIP-style protocols, and chain-neutral governance inputs.

C. Long-Term

Long-term work includes multi-root federation, privacy-preserving Discovery, cross-domain capability governance, independent implementation conformance, formal security analysis, and broader standardization engagement. The goal is to make OAN a credible trust layer for open Agent interconnection rather than a single project demonstration.

29. Conclusion

OAN provides a missing trust-infrastructure layer for open Agent ecosystems. It does not replace Agent interaction protocols, model frameworks, or business workflows. It answers a different set of questions: who is this Agent, who accepted its identity, whether the identity is current, whether a Discovery service is authorized to expose it, and how a peer can validate these facts before interaction.

By separating federated infrastructure governance, Root policy enforcement, Registrar onboarding, CDN distribution, Discovery exposure, and Agent-to-Agent invocation, OAN gives heterogeneous operators a practical way to cooperate. It is a project for making Agent interconnection verifiable, governable, and compatible with the protocol diversity that will inevitably exist in the Agent Internet.

The Root Node's three hub roles make this separation operational. As a trust authorization hub, Root binds accepted identity state and governance-backed evidence. As a data-distribution hub, it turns accepted versions into verifiable packages for downstream indexing. As a semantic-governance hub, it governs the shared tag tree that connects registration-time metadata guidance with Discovery-time semantic matching. The same structure is reflected by the project's public interaction surfaces, SDKs, reference implementation profile, and `did:oan` method specification, all of which expose the trust model without becoming independent authorities.

Acknowledgment

The author thanks Jin Jian, Xie Jiagui, Li Bingqi, and Zhu Runkai for their support.

References

[1] Open Container Initiative, "Open Container Initiative Distribution Specification," https://specs.opencontainers.org/distribution-spec/, 2026, accessed: 2026-06-05.

[2] ——, "Open Container Initiative Image Format Specification," https://specs.opencontainers.org/image-spec/, 2026, accessed: 2026-06-05.

[3] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[5] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, and C. Wu, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o

[6] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

[7] Model Context Protocol Contributors, "Model context protocol specification," https://modelcontextprotocol.io/specification/, 2025.

[8] A2A Protocol Contributors, "Agent2agent protocol specification," https://a2a-protocol.org/latest/specification/, 2025.

[9] Agent Network Protocol Contributors, "Did:wba method specification: Web-based agent decentralized identifier," https://agentnetworkprotocol.com/en/specs/03-did-wba-method-specification/, 2025.

[10] W3C Credentials Community Group, "Decentralized identifiers (dids) v1.0," https://www.w3.org/TR/did-core/, 2022.

[11] W3C Verifiable Credentials Working Group, "Verifiable credentials data model v2.0," https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[12] S. Cheshire and M. Krochmal, "Dns-based service discovery," RFC 6763, 2013.

[13] B. M. Schwartz, M. Bishop, and E. Nygren, "Service binding and parameter specification via the dns (svcb and https resource records)," RFC 9460, 2023.

[14] J. Xu, "Grail: A deep-granularity hybrid resonance framework for real-time agent discovery via slm-enhanced indexing," *arXiv preprint arXiv:2605.02489*, 2026.

[15] Model Context Protocol Contributors, "Model Context Protocol Registry," https://github.com/modelcontextprotocol/registry, 2026, accessed: 2026-06-05.

[16] AGNTCY Contributors, "Open Agentic Schema Framework," https://github.com/agntcy/oasf, 2026, accessed: 2026-06-05.

[17] ——, "Agent Directory Service Overview," https://docs.agntcy.org/dir/overview/, 2026, accessed: 2026-06-05.

[18] K. Huang, V. S. Narajala, I. Habler, and A. Sheriff, "Agent name service (ans): A universal directory for secure ai agent discovery and interoperability," *arXiv preprint arXiv:2505.10609*, 2025.

[19] E. Cui, Y. Cheng, R. She, D. Liu, Z. Liang, M. Guo, T. Li, Q. Wei, W. Xing, and Z. Zhong, "Agentdns: A root domain naming system for llm agents," *arXiv preprint arXiv:2505.22368*, 2025.

[20] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero trust architecture," NIST Special Publication 800-207, 2020.

[21] Cloud Native Computing Foundation, "The spiffe and spire project for workload identity," https://spiffe.io/, 2022.

[22] J. Xu and B. Li, "Darwinnet: An evolutionary network architecture for agent-driven protocol synthesis," *arXiv preprint arXiv:2604.01236*, 2026.

[23] C. Qian, X. Cong, C. Yang, W. Chen, Y. Su, J. Xu, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15 174–15 186.

[24] R. Raskar, P. Chari, J. Zinky, M. Lambe, J. J. Grogan, S. Wang, R. Ranjan, R. Singhal, S. Gupta, R. Lincourt, R. Bala, A. Joshi, A. Singh, A. Chopra, D. Stripelis, B. B, S. Kumar, and M. Gorskikh, "Beyond dns: Unlocking the internet of ai agents via the nanda index and verified agentfacts," *arXiv preprint arXiv:2507.14263*, 2025.